

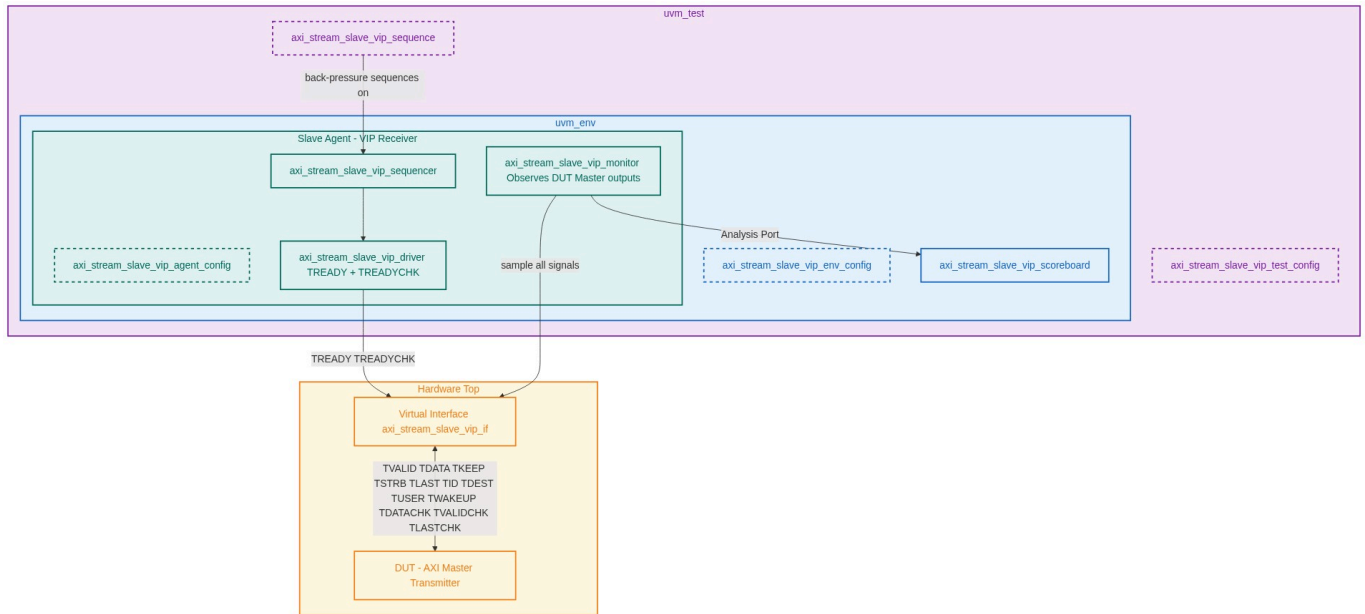
Verification Plan Report

Project: AXI-Stream Slave VIP Verification Plan ? VIP as Receiver (Slave), DUT as Tra

Version: 1.0

Verification Role: Slave

1. UVM Environment Architecture



2. Test Requirements

Index	Feature/Sub	Name	Description
-------	-------------	------	-------------

Verification Plan Report

REQ_SLV_VI P_01	Back-Pressure Control (TREADY Stall Depth and Duty Cycle)	VIP Slave TREADY Back-Pressure Generation	The AXI-Stream Receiver (Slave VIP) is the sole controller of TREADY. Per Section 2.2, a Receiver is permitted to assert TREADY before TVALID (pre-assertion), defer it until after TVALID (back-pressure), or assert it in the same cycle as TVALID (simultaneous). The Slave VIP must implement configurable stall depth (0 to 32 cycles) and duty-cycle randomization to exercise all DUT Master behavioral branches. Section 3.1.1 confirms that if TREADY is absent from the interface, it defaults to HIGH, indicating an always-ready Receiver ? the VIP must explicitly test the non-default (back-pressure) path. Verification Intent: The Slave VIP sequence library must include zero-stall, fixed-stall, and pseudo-random-stall TREADY patterns. For each pattern, the protocol monitor must verify that the DUT Master's TVALID and payload signals remain stable throughout the stall window. Performance metrics (effective throughput, stall-to-transfer ratio) must be logged by the scoreboard for each regression run to characterize the DUT Master's back-pressure tolerance.
--------------------	---	--	--

Verification Plan Report

REQ_SLV_VI P_02	TVALID Enforcement (Non-Retraction TVALID)	Stability of	DUT Master TVALID Asserted-Until-Handshake Invariant	The specification mandates: 'A Transmitter is not permitted to wait until TREADY is asserted before asserting TVALID. Once TVALID is asserted, it must remain asserted until the handshake occurs.' This creates an asymmetric obligation: the Slave VIP can assert and deassert TREADY freely, but the DUT Master must never retract TVALID once driven HIGH. A mid-window TVALID deassertion is a silent failure mode ? it does not produce a deadlock but corrupts the Receiver's expectation state. The VIP monitor must perform cycle-accurate TVALID polarity tracking to detect any violation. Verification Intent: Protocol monitor assertions must fire within one cycle of detecting TVALID=0 during a TVALID=1 pending window (before TREADY is asserted). Negative stimulus scenarios must deliberately inject TVALID glitches at cycles 2, 8, and 16 of a stall window to verify that the monitor's assertion fires deterministically. Scoreboard must distinguish between legitimate TVALID deassertions (after handshake) and illegal ones (before handshake).
--------------------	---	-----------------	--	---

Verification Plan Report

REQ_SLV_VI P_03	Payload Stability Verification (Bit-Stable Hold of All Payload Signals)	DUT Master Payload Stability During Back-Pressure Window	Section 2.2.1 states: 'The valid data bytes and control information from the Transmitter must remain unchanged once TVALID has been asserted.' This obligation covers TDATA, TKEEP, TSTRB, TLAST, TID, TDEST, and TUSER ? the complete payload bundle. Any bit mutation in any of these signals during the TVALID=1/TREADY=0 window constitutes a protocol violation because the Slave VIP has no mechanism to distinguish a 'refresh' from a 'correction' without an explicit handshake. The failure mode is silent data corruption at the destination. Verification Intent: The monitor must capture a bit-accurate snapshot of all payload signals on the rising clock edge where TVALID is first observed HIGH. On each subsequent clock within the pending window, it must XOR the snapshot against the current values and flag any non-zero result as a payload mutation error. Long stall windows (>16 cycles) combined with wide TDATA (32+ bits) and multi-signal payloads provide the highest sensitivity for detecting intermittent DUT Master drive-strength issues.
--------------------	--	---	---

Verification Plan Report

REQ_SLV_VI	Packet	Boundary	DUT	Master	TLAST	Packet
P_04	Tracking (TLAST Delineation Preservation)	Boundary and Count	Boundary Compliance			
TLAST demarcates the terminal beat of each packet. Section 2.6 states: 'The number of packets and the number of assertions of TLAST must be preserved between the Transmitter and Receiver.' The Slave VIP's scoreboard must count every TLAST assertion as a packet boundary event and verify that the DUT Master does not suppress or duplicate TLAST events. The implicit packet start (no TSTART signal in the protocol) must be inferred from the end of the prior packet for each unique TID/TDEST pair. The first packet after reset is a special case: the first transfer with a given TID/TDEST pair is by definition the first beat of a new packet. Verification Intent: Scoreboard must maintain a per-TID/TDEST packet counter. Sequences must cover single-beat packets (TLAST on beat 0), variable-length packets (8, 64, 256 beats), and back-to-back packets with zero idle gap. The monitor must flag TLAST deassertion in a previously-announced-terminal-beat context as a protocol violation. TLAST preservation across constrained-random back-pressure windows confirms that the DUT Master does not gate TLAST based on TREADY state.						

Verification Plan Report

REQ_SLV_VI P_05	Reset Conformance (Post-Reset Assertion Timing)	Protocol TVALID	DUT Master TVALID Suppression and Post-Reset Recovery	Section 2.8.2 mandates: 'During reset, TVALID must be driven LOW. A Transmitter interface must only begin driving TVALID at a rising ACLK edge following a rising edge at which ARESETn is asserted HIGH.' This creates a precise post-reset sequencing rule: the DUT Master must suppress TVALID for a minimum of one ACLK cycle after reset deassertion. Premature TVALID assertion ? even by one cycle ? constitutes a timing violation. Mid-transfer reset injection (ARESETn asserted while TVALID=1) must cause the DUT Master to abort the in-flight transfer by driving TVALID=0 without completing the handshake. Verification Intent: The monitor must track ARESETn level and flag TVALID=1 during any active reset window as a fatal violation. Post-reset, the monitor must record the first ACLK edge where ARESETn=1 and verify that TVALID does not appear before the second rising edge minimum. Reset injection scenarios must cover three contexts: (1) idle, (2) mid-packet pending (TVALID=1, TREADY=0), and (3) at the exact handshake cycle (TVALID=TREADY=1).
--------------------	--	--------------------	--	---

Verification Plan Report

REQ_SLV_VI P_06	Byte Qualifier Reception (TKEEP/TSTRB Encoding Correctness)	Byte Qualifier Semantic Interpretation and Illegal Encoding Detection	TKEEP and TSTRB encode per-byte transport semantics: TKEEP=1/TSTRB=1 is a data byte, TKEEP=1/TSTRB=0 is a positional placeholder, TKEEP=0/TSTRB=0 is a null byte. The combination TKEEP=0/TSTRB=1 is architecturally reserved and constitutes a protocol violation. The Slave VIP's scoreboard must reconstruct the ordered byte stream from received beats, correctly excluding null bytes from the data count. The monitor must assert on detection of any TKEEP=0/TSTRB=1 per-byte combination. Verification Intent: The VIP must generate TREADY patterns that stall transfers across all TKEEP/TSTRB permutations, including the single-active-byte case (TKEEP=0b0001 on a 4-byte bus). The reserved combination must be driven exclusively in negative test scenarios. The scoreboard must verify byte-exact data reconstruction for sparse streams (low TKEEP density) and must confirm that positional bytes (TSTRB=0) are not counted as data bytes.
--------------------	---	---	---

Verification Plan Report

REQ_SLV_VI P_07	TWAKEUP Protocol Adherence (TWAKEUP Pre-TVALID Lead-Time and Persistence)	DUT Master TWAKEUP Timing and Glitch-Free Assertion	Section 2.3 defines the TWAKEUP rules: it must be glitch-free (synchronous to ACLK, suitable for asynchronous sampling); if asserted simultaneously with TVALID, it must persist until TREADY is asserted; a Receiver is permitted to gate TREADY on TWAKEUP, creating a potential deadlock if TWAKEUP is absent. The recommended timing is TWAKEUP asserted at least one cycle before TVALID. The Slave VIP must exercise both the recommended and the permissible simultaneous timing, as well as the degenerate case (TWAKEUP asserts/deasserts without a subsequent transfer). Verification Intent: The VIP monitor must track TWAKEUP edges relative to TVALID assertion and measure the lead-time in ACLK cycles. A watchdog timer must fire if the Slave VIP gates TREADY on TWAKEUP and TWAKEUP is never driven, confirming deadlock detection capability. The simultaneous scenario requires the monitor to verify TWAKEUP=1 on every cycle from the TVALID=TWAKEUP=1 cycle through the first cycle where TREADY=1.
--------------------	---	--	--

Verification Plan Report

REQ_SLV_VI P_08	TID/TDEST Isolation (Per-Stream Delivery Cross-Stream Orthogonality)	Stream In-Order and	Multi-Stream Interleaving with In-Order Verification	Section 2.7 permits interleaving of transfers across distinct TID/TDEST streams at per-transfer granularity, with no constraint to TLAST boundaries. Within any single stream (same TID+TDEST), reordering is prohibited. Section 4.2 specifies transfer ordering: transfers from a given source to a given destination must arrive at the destination in the order they were transmitted. The Slave VIP scoreboard must maintain independent per-stream receive queues, keyed by concatenated TID and TDEST values, and verify that the DUT Master drives beats for each stream in a monotonically advancing order. Verification Intent: Multi-stream sequences must simultaneously generate at least four distinct TID values, interleaved at single-beat granularity. Scoreboard must detect cross-stream contamination (beats mis-routed to the wrong stream queue) and intra-stream reordering (beat N arrives before beat N-1 for the same stream). Back-pressure patterns applied to individual streams must not cause ordering violations in other streams, verifying complete state orthogonality in the VIP's tracking structures.
--------------------	---	---------------------------	---	---

Verification Plan Report

REQ_SLV_VI P_09	Null Packet Handling (Zero-Data TLAST Transfer Reception)	VIP Slave Null Packet Acceptance Without Byte Counter Corruption	A null packet is defined as a transfer where TLAST=1 but all TKEEP bits are deasserted (TKEEP=0...0), yielding zero data or positional bytes. Section 2.6.1 permits this construct for buffer flushing, packet-end signaling, and endpoint synchronization. The Slave VIP must acknowledge the null packet via handshake (assert TREADY) but must not credit any bytes to the null packet's data count. The scoreboard's byte counter for the corresponding TID/TDEST stream must be unchanged after a null packet handshake. Verification Intent: The monitor must classify each received packet as null or non-null based on TKEEP state at the TLAST beat. Scoreboard must maintain an independent null packet event counter per stream and verify that the per-stream byte accumulator is not advanced by null packet handshakes. Back-to-back null packets must be tested without a data packet in between to verify that the packet-boundary state machine does not stall on zero-data transitions.
--------------------	---	---	---

Verification Plan Report

REQ_SLV_VI P_10	AXI5 Parity Compliance (TDATACHK/TVALIDCHK /TLASTCHK Verification and TREADYCHK Generation)	Bidirectional AXI5 Parity ? Reception Verification and VIP Output Generation	Table 5-1 defines the full AXI5-Stream check signal set. The Slave VIP has two distinct parity responsibilities: (1) independently verifying all DUT Master check signals (TDATACHK, TVALIDCHK, TLASTCHK, TWAKEUPCHK ? computed as odd parity over associated data signals per their Check Enable conditions); and (2) generating TREADYCHK on every cycle where ARESETn=1, as TREADY is owned by the Slave. TDATACHK must be verified for ALL byte lanes regardless of TKEEP state, precluding null-byte parity shortcuts. Parity is computed per-byte for TDATACHK (one check bit per 8-bit lane) and per-signal for single-bit signals. Verification Intent: The monitor must independently compute expected parity for every DUT-driven check signal on every eligible cycle and assert equivalence against the received value. The driver must generate correct TREADYCHK on every cycle, including cycles where TREADY transitions ? a transition cycle is one of the highest-risk moments for check-bit errors. Fault injection scenarios must invert individual bits of TDATACHK at specified beat indices to verify that the monitor's parity checker fires within one clock of detection.
--------------------	---	--	---

Verification Plan Report

REQ_SLV_VI P_11	Back-Pressure During Packet Transfer (Mid-Packet Back-Pressure Without Packet Corruption)	Intra-Packet Back-Pressure Application at Arbitrary Beat Positions	The Slave VIP must be capable of asserting back-pressure (TREADY=0) at any beat position within a multi-beat packet ? including the first beat, the penultimate beat before TLAST, and the TLAST beat itself. Section 2.2 makes no distinction between intra-packet and inter-packet stalls; the DUT Master must hold all in-flight signals stable regardless of position. Stalling precisely on the TLAST beat is of particular interest because the DUT Master's internal packet-boundary state machine must not prematurely advance to the next-packet context during the hold window. Verification Intent: Sequences must parameterize the stall position within a packet as a constrained-random variable: first beat, random mid-packet beat, penultimate beat, and terminal (TLAST) beat. The scoreboard must reconstruct the full packet from observed beats, regardless of how many stall cycles were inserted, and verify data integrity end-to-end. The monitor must verify that TLAST polarity does not change during a stall on the TLAST beat.
--------------------	---	--	---

Verification Plan Report

REQ_SLV_VI P_12	Continuous Packets Mode (Continuous_Packets Structural Constraint Enforcement)	DUT Master Continuous_Packets Conformance Monitoring	When Continuous_Packets=True, Section 3.3 imposes three structural constraints on the DUT Master: (1) TID and TDEST must not change while TLAST=0 (no mid-packet stream switching); (2) TSTRB is not present (position bytes not supported); and (3) all TKEEP bits must be HIGH while TLAST=0 (no mid-packet null bytes). Null bytes are permitted only in bytes above the data bytes in the terminal beat (TLAST=1). Interconnect components in this mode must only arbitrate at TLAST boundaries. The Slave VIP monitor must enforce all three constraints in real-time and report the specific constraint violated. Verification Intent: Continuous_Packets mode must be exercised with at least 8 consecutive packets of variable length. The monitor must issue a single-cycle latency assertion for any of the three constraint violations. Negative scenarios must individually violate each constraint: (a) TID change at beat 4 of an 8-beat packet, (b) null byte injection at beat 3, (c) TSTRB asserted in a Continuous_Packets interface. Each negative scenario must produce exactly one assertion with the correct constraint-ID in the error report.
--------------------	---	---	--

3. Test Cases & Mapping

Index	Scenario	TR Mapping
TC_SLV_VIP_001	Zero-Stall Maximum Throughput: VIP Slave pre-asserts TREADY before TVALID arrives. DUT Master drives 256-beat packet; all beats transfer in consecutive cycles with no stall. Scoreboard measures 100% bus utilization (throughput = 1 transfer/cycle) and verifies full payload integrity.	REQ_SLV_VIP_01
TC_SLV_VIP_002	Fixed-Stall Single Back-Pressure Window: After TVALID=1 is observed, VIP deasserts TREADY for exactly 8 cycles. Monitor verifies TVALID and all payload signals remain bit-stable across the 8-cycle window. Transfer completes on TREADY reassertion at cycle 9.	REQ_SLV_VIP_01, REQ_SLV_VIP_03

Verification Plan Report

TC_SLV_VIP_003	Random-Duty-Cycle Back-Pressure: VIP toggles TREADY with a constrained-random duty cycle (10%?70% high) across a 64-beat packet. Beat count and payload integrity verified at end of packet. Throughput metric logged.	REQ_SLV_VIP_01
TC_SLV_VIP_004	TREADY Free-Running Pre-Assertion: VIP asserts and deasserts TREADY multiple times before TVALID appears. Verifies that pre-assertion toggling is legal per spec and that the DUT Master does not misinterpret early TREADY pulses as spurious handshakes.	REQ_SLV_VIP_01
TC_SLV_VIP_005	TVALID Stability Nominal: VIP holds TREADY=0 for 32 cycles after TVALID=1 assertion. Monitor captures payload snapshot on cycle 1 and XOR-checks against all 32 pending cycles. Expects zero XOR diff across all payload signals.	REQ_SLV_VIP_02, REQ_SLV_VIP_03
TC_SLV_VIP_006	TVALID Stability ? Glitch Injection at Cycle 8 (Negative): Simulation model injects a 1-cycle TVALID=0 glitch at cycle 8 of a 16-cycle back-pressure window. VIP monitor must fire a TVALID_RETRACTION_VIOLATION assertion within one clock of the glitch.	REQ_SLV_VIP_02
TC_SLV_VIP_007	TVALID Stability ? Glitch at Cycle 2 (Negative): TVALID=0 injected at cycle 2 of stall ? earliest possible retraction. Confirms monitor assertion fires immediately without delay hysteresis.	REQ_SLV_VIP_02
TC_SLV_VIP_008	Post-Handshake TVALID Deassertion (Positive): After TVALID=TREADY=1 handshake, TVALID legally deasserts. Monitor must NOT fire a violation. Verifies false-positive suppression in the stability checker.	REQ_SLV_VIP_02
TC_SLV_VIP_009	TDATA Payload Mutation Detection (Negative): TDATA[7:0] forced to change on cycle 5 of a 12-cycle stall. Monitor must detect the bit mutation via snapshot XOR comparison and log the specific changed bits.	REQ_SLV_VIP_03
TC_SLV_VIP_010	Full Payload Bundle Stability ? 32-Cycle Stall: VIP stalls for 32 cycles on a single-beat packet. Monitor verifies TDATA, TKEEP, TSTRB, TLAST, TID, TDEST, and TUSER are all bit-stable across the entire 32-cycle window.	REQ_SLV_VIP_03
TC_SLV_VIP_011	Payload Stability With Parity Cross-Verification: During a 16-cycle stall, monitor verifies that TDATACHK also remains stable (as TDATA is stable, parity must remain constant). Detects DUT Master parity logic regeneration timing issues.	REQ_SLV_VIP_03, REQ_SLV_VIP_10
TC_SLV_VIP_012	Single-Beat Packet Boundary: TVALID=1, TLAST=1 on the first and only beat. VIP asserts TREADY in the same cycle. Scoreboard increments packet counter by 1 and verifies byte count equals number of active TKEEP lanes.	REQ_SLV_VIP_04

Verification Plan Report

TC_SLV_VIP_013	Variable-Length Packet ? 64 Beats: 64-beat packet with TLAST=1 on beat 63. VIP applies random back-pressure within the packet. Scoreboard reconstructs the complete 64-beat packet and verifies TLAST count = 1.	REQ_SLV_VIP_04
TC_SLV_VIP_014	Back-to-Back Packets ? Zero Idle Gap: TVALID stays HIGH across TLAST boundary; beat 64 (TLAST=1) is immediately followed by beat 65 (TLAST=0) of a new packet. Scoreboard must count two packets from one TVALID assertion window.	REQ_SLV_VIP_04
TC_SLV_VIP_015	10-Packet Regression ? TLAST Count Audit: DUT Master drives 10 sequential packets of randomized lengths (1?32 beats each). Scoreboard verifies total TLAST count = 10 and that each individual packet was correctly bounded.	REQ_SLV_VIP_04
TC_SLV_VIP_016	TVALID=1 During ARESETn (Negative): TVALID driven HIGH during the last cycle of ARESETn assertion. Monitor must flag TVALID_DURING_RESET_VIOLATION. Verifies reset protocol checker responsiveness.	REQ_SLV_VIP_05
TC_SLV_VIP_017	Post-Reset Minimum Latency: TVALID asserted exactly 1 ACLK cycle after ARESETn deassertion (first legal cycle). Verifies DUT Master meets the minimum post-reset timing without any additional forced delay.	REQ_SLV_VIP_05
TC_SLV_VIP_018	Post-Reset Extended Idle (8 Cycles): DUT Master holds TVALID=0 for 8 cycles after reset deassertion before initiating transfer. Verifies monitor accepts this extended idle window as fully compliant.	REQ_SLV_VIP_05
TC_SLV_VIP_019	Mid-Transfer Reset Injection: During a 16-beat packet (TVALID=1, TREADY=0 back-pressure), ARESETn is driven LOW for 4 cycles. Monitor verifies DUT Master drives TVALID=0 within the reset window. Post-reset, a clean restart is verified.	REQ_SLV_VIP_05
TC_SLV_VIP_020	Reset at Handshake Coincidence: ARESETn driven LOW on the exact cycle where TVALID=TREADY=1. Verifies DUT Master aborts the completing handshake and returns to post-reset idle without spurious completion.	REQ_SLV_VIP_05
TC_SLV_VIP_021	All-Data Byte Lane Qualifier: TKEEP=4'b1111, TSTRB=4'b1111 across 100 beats. Scoreboard confirms 4 data bytes per beat and byte accumulator advances by 4 per handshake.	REQ_SLV_VIP_06
TC_SLV_VIP_022	Sparse Null-Byte Stream: TKEEP constrained to randomized patterns (at least one lane always zero) across 64 beats. Scoreboard reconstructs per-beat active byte counts from TKEEP and verifies total received byte count.	REQ_SLV_VIP_06

Verification Plan Report

TC_SLV_VIP_023	Single Active Byte ? LSB Lane Only: TKEEP=4'b0001, TSTRB=4'b0001 for 16 beats. Scoreboard accumulates exactly 1 byte per beat. Monitor verifies upper three TDATA byte lanes are not counted as data.	REQ_SLV_VIP_06
TC_SLV_VIP_024	Reserved Qualifier Injection (Negative): DUT Master drives TKEEP[1]=0, TSTRB[1]=1 on beat 7 of a 16-beat transfer. Monitor must fire RESERVED_QUALIFIER_VIOLATION within one clock of the illegal beat.	REQ_SLV_VIP_06
TC_SLV_VIP_025	TWAKEUP Recommended Timing ? 2-Cycle Lead: TWAKEUP asserted 2 cycles before TVALID. VIP does not gate TREADY on TWAKEUP. Monitor verifies TWAKEUP lead-time coverage bin = early (2 cycles).	REQ_SLV_VIP_07
TC_SLV_VIP_026	TWAKEUP Simultaneous With TVALID, VIP Stalls 6 Cycles: TWAKEUP=1 and TVALID=1 in the same cycle. VIP holds TREADY=0 for 6 cycles. Monitor verifies TWAKEUP remains HIGH for all 6 stall cycles until TREADY=1 is asserted.	REQ_SLV_VIP_07
TC_SLV_VIP_027	TWAKEUP Without Transfer (Degenerate): TWAKEUP asserted for 4 cycles, then deasserted, without TVALID ever being asserted. Monitor logs event but does not raise violation (this is legal per spec Section 2.3).	REQ_SLV_VIP_07
TC_SLV_VIP_028	TWAKEUP Early Deassertion During Stall (Negative): TWAKEUP deasserted on cycle 3 of a 6-cycle stall where TWAKEUP=TVALID=1 was simultaneous. Monitor must detect TWAKEUP_PREMATURE_DEASSERTION violation.	REQ_SLV_VIP_07
TC_SLV_VIP_029	Dual-Stream Interleaving (TID=0/1): DUT Master alternates beats between TID=0 and TID=1 for 64 beats each (128 total). VIP scoreboard maintains two separate receive queues and verifies in-order delivery for each stream independently.	REQ_SLV_VIP_08
TC_SLV_VIP_030	Four-Stream Interleaving (TID=0/1/2/3): DUT Master cycles through 4 TID values at single-beat granularity for 200 total beats. VIP scoreboard tracks all 4 streams concurrently. Back-pressure applied selectively to verify no cross-stream contamination.	REQ_SLV_VIP_08
TC_SLV_VIP_031	Stream Switch at TLAST Boundary: DUT Master switches TID at every TLAST assertion. Verifies that per-packet stream isolation is preserved even when back-pressure is applied on the TLAST beat.	REQ_SLV_VIP_08
TC_SLV_VIP_032	Mid-Packet TID Change (Negative): DUT Master changes TID on beat 5 of a 10-beat packet (TLAST=0). Monitor must fire STREAM_ID_MUTATION_VIOLATION within one clock of the TID change.	REQ_SLV_VIP_08

Verification Plan Report

TC_SLV_VIP_033	Isolated Null Packet: VIP receives TVALID=1, TLAST=1, TKEEP=4'b0000. VIP asserts TREADY in the same cycle. Scoreboard verifies null-packet counter increments by 1 and data byte counter does not advance.	REQ_SLV_VIP_09
TC_SLV_VIP_034	Back-to-Back Null Packets: Two consecutive null packets with no data packet or idle gap between them. Scoreboard counts 2 null packets. Monitor verifies packet-boundary state machine does not stall.	REQ_SLV_VIP_09
TC_SLV_VIP_035	Null Packet Following Non-Null Packet: 8-beat data packet (TKEEP=4'b1111) immediately followed by a null packet. Scoreboard verifies the data packet's byte count is complete and the null packet adds 0 bytes to the accumulator.	REQ_SLV_VIP_09
TC_SLV_VIP_036	Null Packet Preceding Non-Null Packet: Null packet followed immediately by an 8-beat data packet. Verifies that the null packet does not disrupt the subsequent data packet's byte accumulation.	REQ_SLV_VIP_09
TC_SLV_VIP_037	AXI5 Parity Nominal ? 1000 Beats: DUT Master drives correct TDATACHK, TVALIDCHK, TLASTCHK, and TWAKEUPCHK for all 1000 beats. VIP monitor independently recomputes parity on every beat and asserts zero errors.	REQ_SLV_VIP_10
TC_SLV_VIP_038	TDATACHK Fault Injection ? Beat 50, Bit 0: DUT Master inverts TDATACHK[0] on beat 50. Monitor detects the parity mismatch on TDATACHK[0] at beat 50 and logs PARITY_FAULT with lane index 0.	REQ_SLV_VIP_10
TC_SLV_VIP_039	TREADYCHK Generation by VIP Slave: Across 500 back-pressure events (TREADY toggling), VIP driver generates TREADYCHK on every cycle (ARESETn=1). Scoreboard verifies TREADYCHK = odd_parity(TREADY) on every cycle ? including TREADY transition cycles.	REQ_SLV_VIP_10
TC_SLV_VIP_040	Null Byte Lane Parity Coverage: TKEEP=4'b0101 (lanes 1,3 active; lanes 0,2 null). DUT Master must compute TDATACHK for ALL 4 lanes, including the two null lanes. Monitor verifies parity correctness on null lanes independently.	REQ_SLV_VIP_10
TC_SLV_VIP_041	Back-Pressure on First Beat of Packet: VIP immediately deasserts TREADY when TVALID=1 arrives on beat 1 of a 32-beat packet. Stall lasts 12 cycles. Scoreboard verifies no data loss and the full 32-beat packet is received correctly.	REQ_SLV_VIP_11
TC_SLV_VIP_042	Back-Pressure on TLAST Beat: VIP deasserts TREADY on the specific beat where TLAST=1. Stall lasts 8 cycles before TREADY assertion. Monitor verifies TLAST polarity is stable throughout the stall and the packet count increments exactly once.	REQ_SLV_VIP_11

Verification Plan Report

TC_SLV_VIP_043	Alternating Back-Pressure Every 8 Beats: In a 256-beat packet, VIP deasserts TREADY for 4 cycles every 8 beats. Monitor verifies no payload mutations during any of the 32 stall windows. Total packet received correctly despite 128 stall cycles.	REQ_SLV_VIP_11
TC_SLV_VIP_044	Continuous_Packets Nominal: 8 consecutive packets (12 beats each) transmitted in Continuous_Packets mode. TID/TDEST stable within each packet, all TKEEP=1 in mid-packet beats. Monitor issues zero violations.	REQ_SLV_VIP_12
TC_SLV_VIP_045	Continuous_Packets ? TID Mid-Packet Change (Negative): TID changes on beat 4 of a 10-beat Continuous_Packets packet. Monitor fires CONTINUOUS_PKTS_TID_VIOLATION immediately.	REQ_SLV_VIP_12
TC_SLV_VIP_046	Continuous_Packets ? Null Byte Mid-Packet (Negative): TKEEP[2]=0 injected on beat 6 of a 12-beat Continuous_Packets packet. Monitor fires CONTINUOUS_PKTS_NULL_BYTE_VIOLATION at beat 6.	REQ_SLV_VIP_12

4. Functional Coverage Groups

Group Name	Description
cg_tready_backpressure_depth	TREADY stall depth bins: 0 cycles (immediate), 1-4 (short), 5-16 (medium), 17-32 (long). Cross with packet position bins: first beat, mid-packet, TLAST beat.
cg_tvalid_to_tready_gap	Number of ACLK cycles from TVALID assertion to TREADY assertion: 0 (simultaneous), 1, 2-4, 5-16, 17-32. Verifies VIP exercises the full range of acceptance latency scenarios seen by DUT Masters.
cg_payload_stability_window	Beat count during a single stall window: 1, 2-4, 5-16, 17-32 cycles. Cross with signal bundle changed (TDATA-only, TKEEP-only, combined) ? used only in negative stimulus to confirm monitor sensitivity.
cg_packet_size_distribution	Received packet length in beats: 1 (single), 2-8 (short), 9-64 (mid), 65-256 (long). Cross with back-to-back vs isolated packet categories. Verifies full length coverage from DUT Master.
cg_reset_injection_context	ARESETn injection context: (a) idle (no pending TVALID), (b) mid-packet pending (TVALID=1, TREADY=0), (c) handshake coincidence (TVALID=TREADY=1). Cross with reset duration bins: 1, 2-4, 5-16 cycles.
cg_byte_qualifier_patterns	Per-byte-lane TKEEP/TSTRB encoding coverage on 4-byte bus: all-data (1111/1111), mixed-null (e.g., 1100/1100), single-active (0001/0001), all-null (0000/0000), positional (1010/0000). Reserved combination coverage must remain zero except in negative scenarios.
cg_twakeup_timing	TWAKEUP lead-time bins relative to TVALID: >=2 cycles early (recommended), 1 cycle early, simultaneous (0 cycles). Simultaneous cross with VIP stall depth bins (0, 1-4, 5-16). No-wakeup case tracked separately as a protocol-degenerate event.

Verification Plan Report

cg_stream_interleaving_depth	Active TID count: 1, 2, 4. Consecutive beat count per TID before stream switch: 1 (maximum interleaving), 2-4, 5-16. Cross with back-pressure applied vs not applied to individual streams.
cg_null_packet_context	Null packet occurrence context: isolated, following a data packet, preceding a data packet, back-to-back null packets. Cross with VIP TREADY response: immediate accept (0-stall) vs deferred (1-8 stall cycles).
cg_parity_lane_coverage	Per-byte-lane TDATACHK bit coverage for 4-byte bus: all 4 lanes individually, all lanes active simultaneously, null-lane parity (TKEEP=0). TREADYCHK transition coverage: TREADY=0?1, TREADY=1?0. TVALIDCHK sampled on every ARESETn=1 cycle.
cg_intra_packet_backpressure_position	Stall beat position within a packet: beat 1 (first), beat 2 to N-2 (mid), beat N-1 (penultimate), beat N (TLAST). Verifies that back-pressure at each canonical intra-packet position has been exercised at least once.
cg_continuous_packets_violations	Continuous_Packets constraint violation type: (a) TID change mid-packet, (b) null byte mid-packet, (c) TSTRB present. Each violation type must be triggered at least once in negative scenarios. Nominal (zero-violation) Continuous_Packets run also binned.

5. Interface Signals

Signal Name	Direction	Width	Description
ACLK	Input	1	Global synchronization clock. VIP Slave samples all DUT Master signals on rising edge. VIP output (TREADY, TREADYCHK) changes must occur after the rising edge. (Section 2.8.1, p.2-28)
ARESETn	Input	1	Active-LOW global reset. May be asserted asynchronously. Deassertion must be synchronous to rising ACLK. DUT Master must drive TVALID=0 during reset assertion. (Section 2.8.2, p.2-28)
TVALID	Input	1	Driven by DUT Master. Indicates valid transfer payload on the bus. Once asserted, must remain HIGH until TREADY is also asserted (handshake). VIP monitor checks stability. (Section 2.2, p.2-18)
TREADY	Output	1	Driven by VIP Slave. Indicates the Receiver can accept an inbound transfer. The primary back-pressure mechanism. May be asserted before or after TVALID, or toggled freely without TVALID. (Section 2.2, p.2-18)
TDATA	Input	TDATA_WIDTH (integer x 8 bits)	Payload signal driven by DUT Master. Must remain stable from first TVALID=1 cycle through handshake completion. VIP monitor performs cycle-accurate snapshot comparison. (Section 2.4, p.2-21)

Verification Plan Report

TSTRB	Input	TDATA_WIDTH /8	Byte qualifier driven by DUT Master. Distinguishes data bytes (TSTRB=1) from positional placeholder bytes (TSTRB=0) within an active TKEEP lane. Not present in Continuous_Packets mode. (Section 2.5.2, p.2-24)
TKEEP	Input	TDATA_WIDTH /8	Byte-transport qualifier driven by DUT Master. When deasserted for a lane, that lane carries a null byte. A full TKEEP=0 with TLAST=1 constitutes a null packet. (Section 2.5.1, p.2-24)
TLAST	Input	1	Packet boundary indicator driven by DUT Master. Asserted on the terminal beat of each packet. TLAST count must be preserved end-to-end. VIP scoreboard tracks per-stream TLAST events. (Section 2.6, p.2-26)
TID	Input	TID_WIDTH (recommended ≤ 8 bits)	Stream identifier driven by DUT Master. Enables per-transfer multiplexing of independent data streams. VIP scoreboard keys receive queues by TID+TDEST. Must remain stable within a packet in Continuous_Packets mode. (Section 2.7, p.2-27)
TDEST	Input	TDEST_WIDTH (recommended ≤ 8 bits)	Routing identifier driven by DUT Master. Used with TID to uniquely identify each stream. VIP scoreboard uses TID+TDEST as composite stream key for ordering verification. (Section 2.7, p.2-27)
TUSER	Input	TUSER_WIDTH (recommended: integer multiple of TDATA_WIDTH /8)	User-defined sideband driven by DUT Master, aligned per-byte to TDATA. Must remain stable during back-pressure window alongside other payload signals. Values for null byte lanes are not guaranteed. (Section 2.9, p.2-29)
TWAKEUP	Input	1 (AXI5-Stream only)	Power-domain activity indicator driven by DUT Master. Must be glitch-free. Must be asserted at minimum simultaneously with TVALID; recommended at least 1 cycle before TVALID. Must persist until TREADY if simultaneous. (Section 2.3, p.2-20)
TDATACHK	Input	TDATA_WIDTH /8 (AXI5-Stream only)	Odd parity check field for TDATA driven by DUT Master. One check bit per 8-bit lane. Check Enable: TVALID=1. Must be correct for ALL lanes including null-byte lanes (TKEEP=0). VIP monitor independently computes expected value. (Table 5-1, p.5-48)
TVALIDCHK	Input	1 (AXI5-Stream only)	Odd parity check bit for TVALID driven by DUT Master. Check Enable: ARESETn=1. Driven on every clock cycle while ARESETn is asserted. VIP monitor verifies against independently computed expected value. (Table 5-1, p.5-48)

Verification Plan Report

TLASTCHK	Input	1 (AXI5-Stream only)	Odd parity check bit for TLAST driven by DUT Master. Check Enable: TVALID=1. For a 1-bit signal, odd parity equals logical inversion. VIP monitor checks TLASTCHK = ~TLAST on every TVALID=1 cycle. (Table 5-1, p.5-48)
TREADYCHK	Output	1 (AXI5-Stream only)	Odd parity check bit for TREADY generated by VIP Slave. Check Enable: ARESETn=1. VIP driver must compute and drive TREADYCHK = odd_parity(TREADY) on every eligible cycle, including transition cycles. (Table 5-1, p.5-48)

6. Verification Environment Structure

axi_stream_slave_vip_tb/

```

|-- slave_agent/
|   |-- axi_stream_slave_vip_driver.sv
|   |-- axi_stream_slave_vip_monitor.sv
|   |-- axi_stream_slave_vip_agent.sv
|   |-- axi_stream_slave_vip_sequencer.sv
|   |-- axi_stream_slave_vip_agent_config.sv
|-- env/
|   |-- axi_stream_slave_vip_scoreboard.sv
|   |-- axi_stream_slave_vip_env.sv
|   |-- axi_stream_slave_vip_env_config.sv
|-- sequences/
|   |-- axi_stream_slave_vip_seq_item.sv
|   |-- axi_stream_slave_vip_base_seq.sv
|   |-- axi_stream_slave_vip_test_sequences.sv
|-- top/
|   |-- axi_stream_slave_vip_defines.sv
|   |-- axi_stream_slave_vip_if.sv
|   |-- axi_stream_slave_vip_pkg.sv
|   |-- axi_stream_slave_vip_test.sv
|   |-- axi_stream_slave_vip_tb_top.sv

```

7. SystemVerilog Implementation

axi_stream_slave_vip_defines.sv (Architecture Blueprint)

COMPONENT: Global Defines

MEMBERS:

- `TDATA_WIDTH = 32 (integer x 8)
- `TID_WIDTH = 8 (max recommended by spec)
- `TDEST_WIDTH = 4
- `TUSER_WIDTH = 4
- `HAS_PARITY = 1 (AXI5-Stream mode)
- `HAS_WAKEUP = 1 (AXI5-Stream mode)

Verification Plan Report

- `CLK_PERIOD = 10 (ns)
- `TREADY_DEFAULT_STALL_MAX = 32 (cycles)
- `PAYLOAD_SNAPSHOT_FIELDS = {TDATA, TKEEP, TSTRB, TLAST, TID, TDEST, TUSER}

axi_stream_slave_vip_seq_item.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_seq_item

EXTENDS: uvm_sequence_item

MEMBERS:

- [rand int] tready_stall_cycles (0 to 32 ? back-pressure duration)
- [rand bit] tready_pre_assert (pre-assert TREADY before TVALID)
- [rand int] tready_duty_high (% high in duty-cycle mode, 10-90)
- [int] stall_position (beat index within packet for intra-packet stall)
- [bit] expect_tvalid_violation (for negative test control)

CONSTRAINTS:

- c_stall: tready_stall_cycles inside {[0:32]}
- c_duty: tready_duty_high inside {[10:90]}

METHODS:

- do_copy, do_compare, convert2string for UVM field automation

axi_stream_slave_vip_agent_config.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_agent_config

EXTENDS: uvm_object

MEMBERS:

- [virtual axi_stream_slave_vip_if] vif
- [uvm_active_passive_enum] is_active = UVM_ACTIVE
- [bit] has_parity (enable TREADYCHK generation and parity checking)
- [bit] has_wakeup (enable TWAKEUP monitoring)
- [bit] continuous_pkt_mode (enable Continuous_Packets constraint checking)
- [bit] enable_tracker (enable packet tracker log output)
- [int] watchdog_cycles = 10000 (TREADY deadlock watchdog)

axi_stream_slave_vip_driver.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_driver

EXTENDS: uvm_driver#(axi_stream_slave_vip_seq_item)

CONFIG_REF: axi_stream_slave_vip_agent_config (Source: ConfigDB)

MEMBERS:

- [virtual axi_stream_slave_vip_if] vif

METHODS & LOGIC:

- build_phase: Get vif from ConfigDB. Get agent config.
- run_phase: Forever loop ? get_next_item(req); drive_tready(req); item_done().
- drive_tready(req):
 - If req.tready_pre_assert: assert TREADY immediately via cb_drv.
 - Wait for TVALID=1 (monitor via cb_mon).
 - Stall TREADY=0 for req.tready_stall_cycles cycles (clocking block).
 - Assert TREADY=1. Wait for TVALID=0 (handshake complete).
 - Release TREADY=0 for inter-transfer idle.

Verification Plan Report

- compute_treadychk(): On every ARESETn=1 cycle, drive TREADYCHK = ~TREADY (odd parity of 1-bit signal = its logical inversion).
- Watchdog: If TVALID=1 and TREADY=0 for > watchdog_cycles, UVM_FATAL deadlock.

axi_stream_slave_vip_monitor.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_monitor

EXTENDS: uvm_monitor

CONFIG_REF: axi_stream_slave_vip_agent_config (Source: ConfigDB)

MEMBERS:

- [virtual axi_stream_slave_vip_if] vif
- [uvm_analysis_port#(axi_stream_slave_vip_seq_item)] ap
- [payload_snapshot_t] snapshot (TDATA, TKEEP, TSTRB, TLAST, TID, TDEST, TUSER)
- [bit] snapshot_valid
- [int] tvalid_pending_cycles

METHODS & LOGIC:

- run_phase: Forever sample cb_mon at every posedge ACLK.

TVALID Stability Check:

On first TVALID=1: capture snapshot, set snapshot_valid=1.

On subsequent TVALID=1 (no handshake yet): XOR current vs snapshot.

Non-zero XOR -> UVM_ERROR PAYLOAD_MUTATION.

On TVALID deassert before handshake: UVM_FATAL TVALID_RETRACTION.

Reset Protocol Check:

If ARESETn=0 and TVALID=1: UVM_FATAL TVALID_DURING_RESET.

On ARESETn 0->1: record first eligible TVALID cycle (must be >= cycle+1).

Byte Qualifier Check:

If any bit (TKEEP[i]=0 AND TSTRB[i]=1): UVM_FATAL RESERVED_QUALIFIER.

TWAKEUP Check:

Track TWAKEUP relative to TVALID; if simultaneous, verify TWAKEUP stable until TREADY.

TID Change Check (Continuous_Packets mode):

If TID changes while TLAST=0: UVM_ERROR CONTINUOUS_PKTS_TID_VIOLATION.

AXI5 Parity Check (if has_parity):

Compute expected TDATACHK, TVALIDCHK, TLASTCHK, TREADYCHK.

Assert equivalence; mismatch -> UVM_ERROR PARITY_FAULT.

Coverage Group Sampling: Sample all 12 covergroups per cycle.

On handshake (TVALID=1, TREADY=1): build seq_item, ap.write(item), clear snapshot.

axi_stream_slave_vip_sequencer.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_sequencer

EXTENDS: uvm_sequencer#(axi_stream_slave_vip_seq_item)

MEMBERS: Standard UVM sequencer fields (no custom additions required)

axi_stream_slave_vip_agent.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_agent

EXTENDS: uvm_agent

CONFIG_REF: axi_stream_slave_vip_agent_config

MEMBERS:

Verification Plan Report

```
- [axi_stream_slave_vip_driver]      drv
- [axi_stream_slave_vip_monitor]     mon
- [axi_stream_slave_vip_sequencer]   sqr
- [uvm_analysis_port]                ap (proxied from mon.ap)
```

METHODS & LOGIC:

```
- build_phase: If is_active: create drv, sqr. Always create mon. ConfigDB set vif.
- connect_phase: If is_active: drv.seq_item_port.connect(sqr.seq_item_export).
  ap = mon.ap.
```

axi_stream_slave_vip_scoreboard.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_scoreboard

EXTENDS: uvm_scoreboard

MEMBERS:

```
- [uvm_analysis_imp] slave_export (receives items from monitor)
- [axi_stream_slave_vip_seq_item queue[$]] stream_queues[bit[TID+TDEST]]
- [int] total_beats, total_pkts, null_pkts, parity_errors, ordering_errors, byte_count
- [int] null_pkt_count[TID+TDEST]
```

METHODS & LOGIC:

```
- write(item):
  Key = {item.tid, item.tdest}. Push to stream_queues[key].
  If item.tkeep == 0 and item.tlast: null_pkt_count[key]++. No byte increment.
  Else: byte_count += popcount(item.tkeep).
  If previous item for same key has later sequence number: ordering_errors++.
- check_phase: Report total_beats, total_pkts, null_pkts, byte_count, errors.
```

axi_stream_slave_vip_env_config.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_env_config

EXTENDS: uvm_object

MEMBERS:

```
- [axi_stream_slave_vip_agent_config] agent_cfg
- [bit] enable_scoreboard = 1
- [bit] enable_coverage   = 1
```

axi_stream_slave_vip_env.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_env

EXTENDS: uvm_env

MEMBERS:

```
- [axi_stream_slave_vip_agent]      agent
- [axi_stream_slave_vip_scoreboard] scb
- [axi_stream_slave_vip_env_config] cfg
```

METHODS & LOGIC:

```
- build_phase: Retrieve cfg from ConfigDB. Create agent. Create scb if enabled.
- connect_phase: agent.ap.connect(scb.slave_export).
```

axi_stream_slave_vip_base_seq.sv (Architecture Blueprint)

Verification Plan Report

COMPONENT: axi_stream_slave_vip_base_seq

EXTENDS: uvm_sequence#(axi_stream_slave_vip_seq_item)

MEMBERS:

- [axi_stream_slave_vip_seq_item] req

METHODS & LOGIC:

- body(): start_item(req); req.randomize(); finish_item(req).

Provides unconstrained random TREADY back-pressure stimulus for smoke runs.

axi_stream_slave_vip_test_sequences.sv (Architecture Blueprint)

COMPONENT: 46 test sequences, one per TC_SLV_VIP_001 to TC_SLV_VIP_046

EXTENDS: axi_stream_slave_vip_base_seq

LOGIC per sequence:

- TC_001: tready_stall_cycles=0; tready_pre_assert=1 (zero-stall path)
- TC_002: tready_stall_cycles=8 (fixed single back-pressure)
- TC_003: tready_duty_high=rand[10:70] (duty-cycle back-pressure)
- TC_004: tready_pre_assert=1, stall after=0 (free-running TREADY before TVALID)
- TC_005: tready_stall_cycles=32 (max stall for stability window check)
- TC_006/007: expect_tvalid_violation=1 (negative ? monitor must fire)
- TC_008: tready_stall_cycles=0 after handshake (post-handshake deassertion is legal)
- TC_009/010/011: Long stall windows with payload snapshot cross-checks
- TC_012-015: Packet boundary scenarios with variable TLAST positions
- TC_016-020: Reset injection in idle/mid-packet/handshake contexts
- TC_021-024: Byte qualifier patterns including negative reserved-encoding
- TC_025-028: TWAKEUP timing variants including deadlock and early deassertion
- TC_029-032: Multi-stream interleaving with ordering checks
- TC_033-036: Null packet variants (isolated, b2b, pre/post non-null)
- TC_037-040: AXI5 parity nominal, fault injection, TREADYCHK generation
- TC_041-043: Intra-packet back-pressure at first/TLAST/alternating positions
- TC_044-046: Continuous_Packets nominal and negative constraint violations

axi_stream_slave_vip_test.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_base_test (extends uvm_test) +

46 concrete test classes via macro unroll

MACRO: AXI_STREAM_SLAVE_VIP_TEST_UNROLL(ID) expands to class for TC_SLV_VIP_<ID>

LOGIC:

- build_phase: Create env. Set vif in ConfigDB. Configure agent_cfg params.
- run_phase: start the corresponding sequence on agent.sqr.

axi_stream_slave_vip_if.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_if (SystemVerilog interface)

MEMBERS: ACLK (port), ARESETn, TVALID, TREADY, TDATA, TSTRB, TKEEP, TLAST, TID, TDEST, TUSER, TWAKEUP, TDATACHK, TVALIDCHK, TLASTCHK, TREADYCHK

CLOCKING BLOCKS:

- cb_drv: output TREADY, TREADYCHK; input TVALID, TDATA, TKEEP, TSTRB, TLAST, TID, TDEST, TUSER, TWAKEUP, TDATACHK, TVALIDCHK, TLASTCHK
- cb_mon: input all signals (observe-only for protocol checking)

Verification Plan Report

axi_stream_slave_vip_pkg.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_pkg (SystemVerilog package)

INCLUDES (in dependency order):

- axi_stream_slave_vip_seq_item.sv
- axi_stream_slave_vip_agent_config.sv
- axi_stream_slave_vip_env_config.sv
- axi_stream_slave_vip_sequencer.sv
- axi_stream_slave_vip_driver.sv
- axi_stream_slave_vip_monitor.sv
- axi_stream_slave_vip_agent.sv
- axi_stream_slave_vip_scoreboard.sv
- axi_stream_slave_vip_env.sv
- axi_stream_slave_vip_base_seq.sv
- axi_stream_slave_vip_test_sequences.sv
- axi_stream_slave_vip_test.sv

axi_stream_slave_vip_tb_top.sv (Architecture Blueprint)

COMPONENT: axi_stream_slave_vip_tb_top (SystemVerilog module)

MEMBERS:

- Clock generator: reg ACLK; initial #5 forever #(`CLK_PERIOD/2) ACLK=~ACLK
- axi_stream_slave_vip_if vif (ACLK)
- DUT instance (axi_stream_master_dut) connected to vif.*
- UVM config set: uvm_config_db#(virtual axi_stream_slave_vip_if)::set(vif)
- run_test() in initial block

RESET: vif.ARESETn driven LOW for 5 cycles, then synchronously released HIGH